Characteristics of Decision-Making During Coding

Paul Ralph
Department of Computer Science
University of Auckland
Auckland, New Zealand
p.ralph@auckland.ac.nz

Ewan Tempero
Department of Computer Science
University of Auckland
Auckland, New Zealand
e.tempero@auckland.ac.nz

ABSTRACT
Code appears replete with decisions including logic, organization, presentation and library use. But are developers consciously making these decisions or does it only look that way post hoc? Analysis of 104 developer interviews indicates that developers make many decisions concerning design, architecture, scope and technology use. However, they struggle to reconstruct code-focused choices between two or more alternatives. This suggests that decision-making is not the dominant cognitive process underlying coding behavior, which raises questions about how we teach coding and how we evaluate coding performance.

Categories and Subject Descriptors
H.1.2 [**User/Machine Systems**]: Human Factors & Software Psychology; D.2.10 [**Software Engineering**]: Design; D.2.11 [**Software Engineering**]: Software Architectures;

General Terms
Design, Human Factors, Theory, Legal Aspects.

Keywords
Decisions, Programming, Choice Architecture, Interviews

1. INTRODUCTION
> *You're making decisions a lot, these micro-decisions all the time and they're just hard to recall examples of, and just because they're everywhere and nowhere*—Interviewee 12

Code appears full of decisions—a while loop instead of a for loop or recursion; a variable named "counter" instead of "index" or "i"; an array instead of list or a heap. Some decisions are made *a priori* (e.g. in a planning meeting); some decisions are made by someone else (e.g. a client); some decisions are made explicitly or implicitly in a design specification or outsourcing contract. Others are made by programmers during coding. They may stem from clear (or biased) reasoning about a defined set of alternatives. Some decisions may be instinctive, arbitrary or obvious consequences of previous decisions or existing architectures.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

EASE '16, June 01 - 03, 2016, Limerick, Ireland
Copyright is held by the authors. Publication rights licensed to ACM.
ACM 978-1-4503-3691-8/16/06 $15.00
DOI: http://dx.doi.org/10.1145/2915970.2915990

In fact, code properties that have logical alternatives do not necessarily indicate that *anyone* decided them. Coders may unconsciously implement code features without considering alternatives. This raises myriad questions, including:

- What kinds of decisions are made during coding?
- What factors influence programmers' decision-making?
- What constrains typical coding decisions?
- Is decision-making an appropriate theoretical lens for understanding coding?

Some research investigates decision-making in software design (e.g. [31, 32]; see below); however, we are unaware of any research that specifically focuses on decisions made in coding contexts. We therefore designed and conducted an interview-based study to investigate the kinds of decisions programmers make while coding.

For the purpose of this study, *coding* (or *programming*) refers to the process of creating source code including writing code, thinking about what to write, looking up examples, reading forums, debugging, checking documentation, consulting colleagues and other support activities that are tightly coupled with creating source code. Coding does **not** denote a phase in an idealized development lifecycle. *Coding* also refers to labelling qualitative data—below, the intended meaning should be clear from the context. Meanwhile, a decision is "The action, fact, or process of arriving at a conclusion regarding a matter under consideration; the action or fact of making up one's mind as to an opinion, course of action, etc." [35]. In other words, a decision is a conscious choice between two or more options. Not all intentional actions entail decisions.

We proceed by describing our two-pronged research methodology (Section 2), followed by the results of semi-automated (Section 3) and manual (Section 4) data analysis. Section 5 triangulates across the two data analysis approaches and discusses the implications of the research. Section 6 summarizes the key message of the paper.

2. METHODOLOGY
Briefly, we analyzed interviews of software developers both manually and using a semi-automated content analysis technique. Being an exploratory study, no *a priori* hypotheses are stated. Instead, we describe how the data was analyzed and highlight the chain of evidence between the text and the findings.

2.1 Sampling and Data Collection
A convenience sample of 104 software developers were interviewed, in English, between March and May of 2015 (Table 1). Interviews were conducted face-to-face (65), via audio/video conference (36) or via telephone (3). Participants were mostly males (86%) with bachelor's degrees (70.2%), who live in New Zealand (74%) and work in the software and IT industry (64%).

Although we did not ask explicitly, eight interviewees volunteered that their tertiary education was in an unrelated field (e.g. business,

Table 1. Interviewee Summary

Dimension	Attribute	Freq.	%
Gender	Female	18	17.3
	Male	86	82.7
Education	Secondary Diploma	2	1.9
	Some Tertiary	2	1.9
	College Diploma	4	3.8
	Bachelor's Degree	73	70.2
	Master's Degree or Postgraduate Diploma	23	22.1
Country	New Zealand	77	74.0
	China	11	10.6
	India	5	4.8
	USA	5	4.8
	Australia	2	1.9
	Other	4	3.8
Industry	Software and IT	67	64.4
	Health	9	6.7
	Business, Marketing & Sales	6	5.8
	Finance	6	5.8
	Security	4	3.8
	Education	2	1.9
	Research & Data Analysis	2	1.9
	other	8	7.7
Interview Style	Face-to-Face	65	62.5
	A/V conference	36	34.6
	Telephone	3	2.9

fine arts, physics). Many participants work for small to medium-sized companies; however, several work for larger firms including Atlassian, Google, Infotech, NEC, Oracle, Tata and Twitter. Interviewees had a mean experience of 2.7 years in their current job and 6.9 years in their current industry (Figures 1-2).

2.2 Data Cleaning

The interviews were transcribed word for word from audio recordings and the transcripts were cleaned using a script we wrote to reduce artifacts that might hinder semi-automated analysis, fix speaker mislabelling and remove odd formatting. Violations were flagged and manually examined. Remedial transformations were determined on a per-transcript basis and added to the script. All analysis was done on the cleaned scripts.

We did not correct interviewees' grammar, malapropisms, etc. or remove verbal static (e.g. "umm", "ahh"). All quotations in this paper reflect, as closely as possible, exactly what participants said.

2.3 Semi-Automated Theme Extraction

To explore the text, we used Leximancer (version 4.00.25.L4, build 13558), a semi-automatic content analysis application. We chose Leximancer because it infers categories directly from the corpus

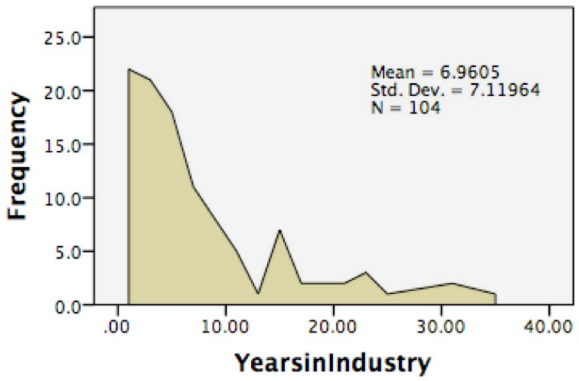


Figure 1. Participant Experience in Current Industry.

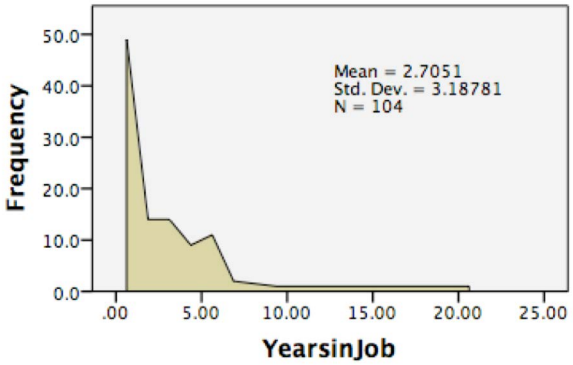


Figure 2. Participant Experience in Current Job.

and has been used effectively on similar data [20]. Leximancer uses unsupervised semantic mapping:

> *A unified body of text is examined to select a ranked list of important lexical terms on the basis of word frequency and co-occurrence usage. These terms then seed a bootstrapping thesaurus builder, which learns a set of classifiers from the text by iteratively extending the seed word definitions. The resulting weighted term classifiers are then referred to as concepts. Next, the text is classified using these concepts at a high resolution, which is normally every three sentences. This produces a concept index for the text and a concept co-occurrence matrix. By calculating the relative co-occurrence frequencies of the concepts, an asymmetric co-occurrence matrix is obtained. This matrix is used to produce a two-dimensional concept map via a novel emergent clustering algorithm. The connectedness of each concept in this semantic network is employed to generate a third hierarchical dimension, which displays the more general parent concepts at the higher levels*. [24]

In other words, Leximancer attempts to answer the question: *what is this text about?* It extracts concepts from unstructured text by examining word frequencies and co-occurrence. It then produces a tree, where nodes represent terms and edges represent semantic connections between terms. This tree is an attempt to visualize the most important terms, their semantic distance and the most important connections between terms. It then clusters the terms to identify parent concepts. This tree is called a *concept map* and the parent concepts are called *themes*.

We initially ran Leximancer using default settings (e.g. default stoplist). The resulting concept map was promising but (as

expected) contained unhelpful terms (e.g. *things*) and synonyms (e.g. *coding* and *programing*), as well as omitting frequent terms that appeared relevant (e.g. *bugs*, *code*). We therefore refined the concept map by:

1. Manually reviewing the stoplist (a list of common words, like *the*, that rarely add semantic information). No changes to the stoplist were deemed necessary.
2. Manually reviewing the complete list of frequent words and names, and adding semantically valuable terms (e.g. *architecture*, *customer*).
3. Excluding interviewer speech because interviewers' repeated usage of certain words (e.g. *example*, *time*) obscured their meaning in the interviewee comments.
4. Iterating between merging closely-related concepts, deleting irrelevant concepts and reviewing the concept map. To determine relevance and relationships, we reviewed text excerpts related to each concept using Leximancer's query features. We stopped—after much iteration—when the concept map appeared to have no more synonyms or irrelevant concepts.
5. Iterating between manipulating the theme size parameter and interpreting themes to generate the most defensible set of themes.
6. Renaming themes better to reflect their content and meaning.

The resulting themes use a theme size of 34% (at 100%, one theme contains all concepts, at 1% each concept has its own theme). We used several techniques to interpret the themes, including:

- Query-based text exploration, that is, using Leximancer queries to explore themes, concepts and relationships.
- Automated summaries, that is, reading the automated text summaries generated by Leximancer.
- Sentiment analysis (using Leximancer), where interviewee attitudes toward a concept are investigated using predefined lists of favorable, unfavorable and negation terms (e.g. *nice*, *bad*, *not*).

Concerning reliability, our iterative refinement suggests that the concept map is fairly robust against minor changes in included concepts. The thematic map is more sensitive to changes in concepts and the theme size parameter. More connected themes appear more robust than less connected themes.

2.4 Manual Coding

We initially intended to use open coding [21] to inductively generate a taxonomy of decisions (e.g. naming, logic, organization). To determine whether this was viable, we analysed 10 randomly-chosen transcripts. We quickly found that interviewees rarely articulated choices between several alternatives, and often went beyond *coding* decisions. It appeared unlikely that open coding would produce the kind of decision taxonomy we expected. Based on this pilot analysis, we broadened our analysis to include decision (e.g. system scope) and choice architecture (e.g. choose one of two alternatives). *Choice architecture* usually refers to how choices are presented to consumers [27]; however, "it also has implications for how decision-makers successively explore the option space, both in choosing what information to examine and what information to ignore as they narrow down their choice set" [9]. That is, just as choice architecture affects how a consumer explores the option space and chooses an option, it affects how a developer explores a design space and chooses a solution [8, 15, 17].

We therefore developed a simple framework to record kinds of decisions (e.g. system scope) and choice architecture (e.g. choose one of two alternatives). As the analysis progressed, we modified the framework better to reflect the salient categories in the data. However, with such a straightforward framework, few modifications were needed.

While interpretation of semi-automated semantic analysis results relies on query-based examination of the text, interpretation of manual coding relies on a natural beginning-to-end reading of transcripts. Reasoning that longer transcripts were more likely to contain deeper discussions, we manually analysed the transcripts from longest to shortest. We continued reading and analyzing the transcripts until it was clear that the marginal utility of an additional transcript was minimal. This saturation point was reached by the 29th transcript (~215 000 words or 43% of the total text).

3. RESULTS OF SEMI-AUTOMATED ANALYSIS

Semi-automatic content analysis produced the concept map (Table 2; Figure 3) and nine significant themes (Table 3; Figure 4). As these figures show the same spanning tree they can be visually compared to see which concepts are in which themes (e.g. the concepts *write* and *test* are in the **code** theme). In the concept map, the size of a node indicates its importance, while edges show the most direct connections between concepts. The proximity of nodes indicates a combination of direct and indirect connectivity. The thematic map follows the same conventions except that importance is indicated by color, not size. More important, more connected concepts have hotter colors (i.e. red, orange) while less important, less connected concepts have cooler colors (i.e. blue, purple). The size of a theme bubble simply indicates which concepts it contains. This section describes the meaning and interpretation of each theme.

(Below, short quotations are presented inline while long quotations are presented in separate italicized paragraphs, following MLA formatting conventions, and Interviewee 00 is abbreviated I00.)

3.1 Theme 1: Decision-Making

Decision-Making the most connected theme and includes *decision*, the most connected concept. This is both unsurprising and reassuring—since the interviews are about decisions, *decision* and **decision-making** should be the central concept and theme.

Decision-Making includes the concepts of *logic* and *architecture*. This indicates that participants make decisions about systems' logic and architecture. For example:

> *Why not just go 'npm install average' and then call require dot average and pass it your array of numbers? So, it's almost about the same amount of typing but it moves that logic completely out of your application. So now … when you read through your code, you just see this line of code is doing an average. I don't need to know how it does it, I know it's doing it.* (I22)

> *If the business logic is over here and there is some way the business logic talks to the UI without being really tightly tied to it then you can switch out the UI in a heartbeat without a lot of that work. So deciding where a piece of code belongs in that tier of architecture I think is a very critical decision that could really come back to haunt you if a bad decision was made.* (I31)

Decision-Making also contains *user* and *customer*, indicating that programmers consider users and customers when making decisions. For example, participants explained, "I always worry about the user experience" (I9) and "Do you need to notify the user to return the unit? And that itself has impacts on customers and down the chain" (I99).

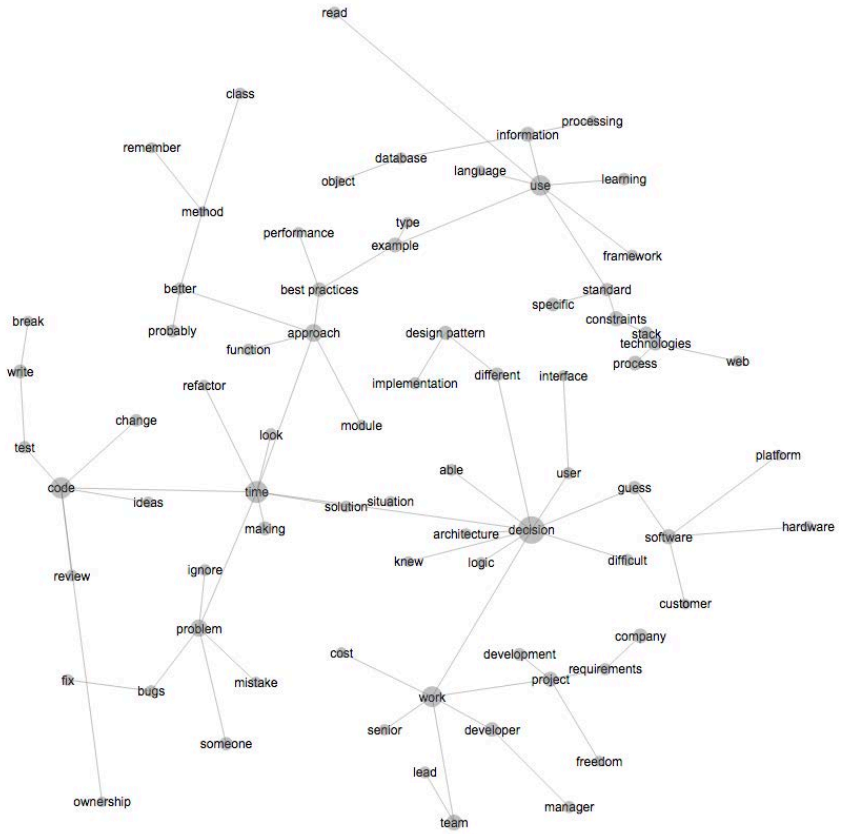


Figure 3. Concept Map

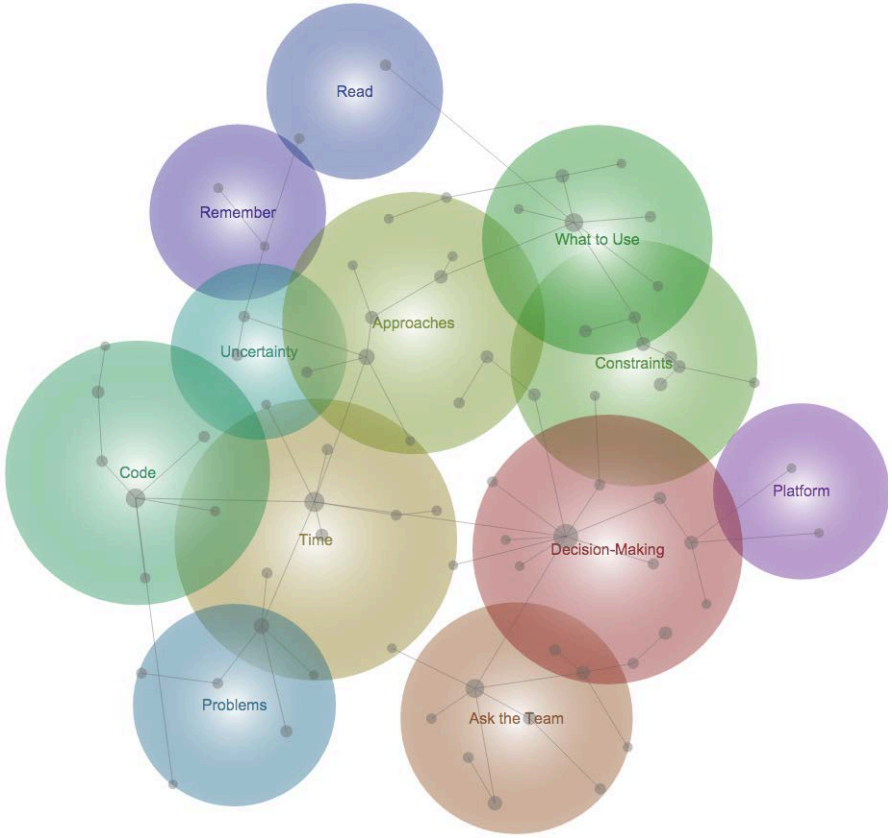


Figure 4. Thematic Map.

Table 2. Most Frequent Concepts

Count	Concept	Count	Concept
2360	Decision	714	Team
1595	Time	650	Information
1509	Code	620	Project
1409	Work	590	Software
1344	Use	571	Example
962	Problem	541	Company
951	Approach	523	Best Practices

Table 3. Top Nine Themes

Theme	Connectivity	Informal Meaning
Decision-Making	100%	Choices made by developers
Ask the Team	85%	Consulting colleagues to make decisions
Time	77%	Time pressure affects decisions
Approaches	69%	Developers decide how to go about addressing problems
Constraints	63%	Developers are constrained by processes, coding standards and technologies
What to Use	63%	Developers sometimes decide which tools, technologies, languages and libraries to use
Code	58%	Developers' decisions are code-focused
Uncertainty	19%	Developers have difficulty articulating the logic behind their decisions
Problems	19%	Many decisions focus on solving problems

3.2 Theme 2: Ask the Team

Many interviewees emphasize consulting colleagues—including managers, team leads and senior developers—to make decisions. For example:

- "When a job is assigned to you and if you can't do it of course you can ask the manager or other senior developers" (I10).
- "I go to more experienced developers, senior developer and then if it's something that can't be answered by them, then I go to project manager" (I33).
- "If I get stuck, I'll go to my team leader" (I3).

Ask the Team also contains the concept of *freedom*. Most interviewees feel ample freedom to build and to refactor their products as they see fit. Sentiment analysis estimates that 71% of references to *freedom* are positive. However, some interviewees felt that their freedom was tied to seniority—"as a junior developer we didn't have much freedom because we have to work according to lead and design but for senior developer have some freedom and they can talk with architect and change code" (I43).

3.3 Theme 3: Time

Time is the second-highest ranked concept and is highly collocated with *Decision* (429 juxtapositions). Participants appear extremely concerned with time:

- **Being on time:** "So that information was lacking and it was really big difficulty for me to deliver my deliverables on time and had to stretch to understand and read more and more about the software and the project requirements" (I92).
- **Saving time:** "So if they comment, it is much easier, it saves time" (I32).
- **Making time:** "If it's major we usually make time to fix it, even if it means adjusting the deadline" (I16)
- **Wasting time:** "So when we tried to do angularish things, it didn't work, and so it took a lot of time, and wasted a lot of, you know, resources and so we thought 'you know what, screw it, let's dump this and go something else'" (I21).
- **Allocating time:** "You spend more time thinking about how to do it as opposed to just 'Hey let's see if this works.'" (I12).
- **Estimating time:** "I spent so long on it and I mean if we had dates from the start you know, we could have easily put a lot more features which utilized that ability to query and yeah we didn't" (I13).

3.4 Theme 4: Approaches

Interviewees indicated that many of their decisions concern what approach to take in building a system or solving a problem. Some participants generally discussed the ongoing quest for better ways of working:

> *Usually in general when I'm asking about whether this is the right way to do something from other people. If they're like, 'no, you can't do it like that', usually that's followed by a discussion about what a better way to do it is. So its not like 'You can't do it like that so go fix it' so much as 'How else could we do it?'* (I24)

Other participants emphasized making decisions about more specific coding approaches including best practices and design patterns. These were both controversial.

Some interviewees felt they were constantly following best practices—"you have the common approach of structuring your application code by creating interfaces, deriving new classes from those and keeping it as loosely coupled as possible. SO yeah, I guess you always work with best practices" (I1). However, others felt that best practices were often counterproductive, e.g., because your legacy system is no longer standards compliant. One participant explained: "Yeah, recently in a code review that I had it was brought to my attention that rather than passing strings by reference for error handing, I can throw exceptions with the specified message. But because I had seen it been done in other places, I assumed it was best practice and decided to follow it." (I4). Another admitted, "sometimes you just get lazy and forget" (I2).

Many interviewees confirmed that they decide which design patterns to use, if any—"I'm making decisions on how the whole solution is structured, what design patterns to use, usually MVC, that kind of thing" (I16). Others have "just sort of been sneaking them in" (I47). Others choose technologies in part because they follow design patterns, e.g. "I like the modular architecture of NodeJS. Ah, that's definitely a design pattern" (I22). Design patterns may even be used unconsciously—"there is a lot of decisions that people come to that might be design patterns even without realising it" (I28). However, some interviewees felt design patterns were overused or used inappropriately. One explained:

> *The team lead who had been there was big on design patterns. I think a lot of the time he fit the problem to the design pattern, as opposed to picking a design pattern to address the problem and as we moved forward and we had to completely re-architect project that we had started, we had to make some decisions on whether we spent time trying to understand what he had done compared to knowing the functionality that was needed and maybe doing a little bit of rework and at that point we decided to do the rework mostly because the performance of what he had come up with was terrible.* (I31)

3.5 Theme 5: Constraints

Participants indicated that their decision-making is constrained by numerous factors including process, standards, frameworks and their technology stack (see also Section 4.1.5). Some interviewees found their organization's development process frustrating—"its frustrating to see something you want to fix but not be able to fix it ... we have processes that say you are not allowed to go and clean up whether or not you like it; you have to have tasks to do it." Others suggested adopting a process is more complicated that just deciding to use it:

> *Basically all the software companies I know attempt to use Scrum … but most of the people in the company ... don't actually know what it means to be Agile, about getting quick feedback and fast iterations and … emergent design…. We're kind of stuck with, basically old people who have done it waterfall style ... so basically what happens is we say we're agile and we do sprints and then there are people basically 'saying why isn't there documentation for this?' Why don't I have a full specification list for this thing?' And I've gotta go 'well that's not the point.'* (I38)

Some interviewees felt constrained by coding standards—"I can directly call the database from UI. That would save me a lot of coding lines. But we are not supposed to do that and go through layers. I had to do it as per … the coding standards." (I39). Others like coding standards—"Usually our coding standards are really good and they don't constrain us" (I85), while others said they had no coding standards at all. One participant admitted, "Sometimes I break conventions, just because it looks cleaner" (I79).

Feelings toward technology stacks were similarly mixed—"technology will be one of the biggest things, um, that will road block us" (I5) vs. "It is not a source of frustration, every technology has its own advantages and disadvantages" (I9). Decisions about

which technology stack to use are often outside an individual developer's authority:

> *At my previous organisation we were kind of an open source Java shop, but unfortunately we never really have the skills, or really good skills in that area, and we were always keen to go the Microsoft .Net kind of way just because the tools were slightly better and it would have just have been faster for us to do stuff that way.* (I65)

Likewise, using different frameworks could constrain or facilitate coding and decision-making. For example, I27 explained, "picking a framework and running with it and then having it lose support or, yeah it just being too much of a black box, and like you don't really know what's going on in there" (I27). In contrast, Interviewee 18 said, "we use ember.js for the framework thing, and pretty much I write something in like a hundred lines, then I can condense this into fifty lines, then I can condense it into twenty lines" (I18).

3.6 Theme 6: What to Use

Developers also make decisions about things to use, including:

- **Frameworks**: "Another option was to rewrite the app with the new features in a complete new framework but then we would end up with one great big app in a completely new framework and all the other ones in an old version of an old framework…. In hindsight … we should've switched completely. Today you can see that the framework we didn't go with, their upgrades are more backwards compatible and they have put more thought into the changes for the framework" (I32).
- **Libraries**: "choosing between two libraries" (I5)
- **Tools**: "So there are many choices we had, GitHub, Fabricator, Review Board, etc." (I15); "There are heaps of plug-ins and frames for you to choose" (I98);
- **Languages**: Some developers decided which language to use—"my first decision was what language to use" (I22), while others did not—"well it's not my decision" (I6).
- **Open-source options**: "whether using an open-source widget or lib developed by others or building one by yourself" (I17)

What to Use also contains the *learning* and *information* concepts. Several participants explained that their decision-making is often constrained by lack of information, requiring learning. For example—"Sometimes we deal with a large client who has multiple departments and it's a bit of a mission getting the information we need" (I16); "there were times when I couldn't proceed further without adequate information" (I34).

3.7 Theme 7: Code

Developers often spoke about **Code**—writing code, changing code, testing code, breaking code and reviewing code (see also Section 4.1.1). The *code* concept is highly collocated with *decisions* (350 juxtapositions). Some kinds of coding decisions not covered elsewhere in this paper include:

- Deciding whether to use configuration or hardcoded values to hide or show fields for specific clients (I33)
- Deciding whether to proceed without testing a module or wait until another module (on which it depends) was implemented (I102)
- Deciding where to focus unit testing (I99)
- Decisions that are informed by code comments (I52)

3.8 Theme 8: Uncertainty

The **Uncertainty** theme reflects numerous interviewee comments suggesting doubts about their decision-making and inability to explain their actions, decisions, motives or logic.

> *For example, in well-established practices you'll have code review. Someone might look at it and go 'Why the hell have you done this?' And you're like 'To be honest, I don't know. Uhm, I, I probably should have stopped and thought about that.' And, it's not necessarily the wrong decision, uhm, it's just something that maybe doesn't quite make sense or maybe it's an amazing decision but you have no idea why you made it.* (I22)

3.9 Theme 9: Problems

The **Problems** theme includes numerous comments about the interaction between decisions and fixing bugs, problems and mistakes. For example, "if you do a certain sequence of events it causes a bug … and we know that only a small percentage of people ever do it, and fixing it is going to take a fair bit of time, then we just decide not to, for now. It's always for now. It's always like, 'Oh, we'll put that on the later pile'" (I27).

Sometimes developers decide that "this is not my issue" (I35) and leave it to someone else. Others emphasized that problems are rarely unique: "before taking other peoples' time, you could take advantage of Google or similar, cause … you're not the only person in the world who's run into a particular problem" (I28).

3.10 Other Observations

Some concepts were interesting despite not clustering into a substantial theme. Briefly, many interviewees described basing decisions on things that they had *read*—"Most of the time you have a problem, you go online and read about it and you can solve your problem after reading online. Most of the time you won't know the solution until you look online. There are sometimes many solutions and it's up to you to decide which one to use" (I55). Stack Overflow (stackoverflow.com) was singled out by approximately one quarter of the participants.

Another concept, *remember*, reflects difficulties recalling specific example of decisions; e.g., "I can't remember specifics" (I56), "I cannot remember" (I43) and "not that I can remember right now" (I1). Asked to describe a decision that turned out well, another participant explained, "If something doesn't break then it is a good decision right? Nobody goes back and thinks 'what is that decision?'" (I15).

4. RESULTS OF MANUAL ANALYSIS

As noted in Section 2, our goal was to identify kinds of decisions developers made. This section therefore categorizes decisions according to the aspects of software development the decisions concerned (kind) and how participants described the options they were choose between (choice architecture). Although interview questions centered on *code-focused* decisions between *two or more alternatives*, only about 1/4 of the decisions described focused on coding and even fewer involved two or more options (Table 4).

4.1 Kinds of Decisions

4.1.1 *Developers Make Implementation Decisions*

Participants described numerous low-level decisions that manifest in source code. These include type to use—"whether something should be an int or a, um... or a float, or a double" (I41), what algorithm to use—"so I opened up my old algorithms book, y'know, looked up y'know, Dijkstra's algorithm, things like that" (I63), dealing with performance issues—"you've either got something that's a bit slow, or you've got something that uses more memory than you would like, and both of those can be bad for the user, so ... that can be a difficult decision sometimes" (I44), or deciding what names to use—"So that's one sort of decision we haven't

Table 4. Frequency Crosstabulation of Observed Decision Types and Choice Architecture.

	Choice Architecture				
Kind	> 2	2	Action	Dialectic	Total
Scope	2%	0%	8%	3%	13%
Design	2%	6%	16%	14%	38%
Architecture	0%	4%	5%	2%	11%
Environment	1%	2%	8%	2%	13%
Implementation	2%	3%	13%	7%	25%
Total	7%	15%	50%	28%	100%

Note: These numbers should be interpreted with caution not only because we used convenience and purposive sampling but also because the distinctions between scope, design, architecture and implementation are subjective.

talked about is that like well how do I go about naming things? It's something I spend a lot of time doing" (I80).

This category captures most of those decisions that we were originally interested in, that is, low-level decisions that manifest directly in source code.

4.1.2 *Developers Make Design Decisions*

By far the most frequent kind of decision discussed by participants was what we classified as "design decisions". While the lines between implementation, design and architecture are subjective, we classified decisions as design if the interviewee appeared to consider it such, the decision had a local impact or it did not more clearly fit with implementation or architecture.

Examples include what design philosophy to consider—"Yes, it is about what to choose between inheritance and composition" (I102), or what data structure to use—"if you have an array, you can jump straight to the key that you want; if you have a linked list, you have to iterate through the entire list, until you get to the key that you want; or if you have a binary tree or a hash table, you can get the key that you want faster, but you still have to iterate through a few things" (I41).

The technological environment (see Section 4.1.5) could provide an option, such as a framework supplied by a language's software development kit (e.g. the Java Collections Framework)—"and I think a good decision was to make leverage of Comparable interface" (I86). A number of comments indicated a need for a design that addressed particular quality attributes—"I tend … to try and make it as extensible as possible" (I99); in particular to resolve potential maintainability issues—"That's butt ugly … and I know that I need to make it better, but do I sit, do I spend the time to make it better to make it more readable for the next person to come along or do I carry on?" (I22).

4.1.3 *Developers Make Architectural Decisions*

Architectural decisions are those that affect a large part of the system. We classified a decision as architectural if it appeared to have widespread impact.

In some cases the decision was simply whether to plan the architecture—"we, ummm sat down and we worked out a rough overall architecture for what it would adhere to" (I77)—or not—"it would have been architected properly from the ground up, rather than just what it is, which is just, yeah, it's all... been... a huge blocks of patchwork, lots of things just added on, and like huge copying and pasting of code" (I41). Other interviewees gave examples of classic architectural decisions, including whether to use push or pull information flow—"So, whether a particular module emits an event when it has new data, or whether you... poll" (I99). Sometimes the architecture decision was a choice of technology environment; for example, "So instead of writing lets say a custom calculation agent that would try to consider every possible mathematical function, we've basically decided to use Excel" (I78).

4.1.4 *Developers Make Scoping Decisions*

Another category of decisions interviewees discussed concerned what the software system should do. Many interviewees saw these as decisions that they should not or do not normally make. In some cases, these decisions arise due to under-specified requirements—"there wasn't any specs on what it should look like and I tried to find out what it looked like and they were basically just saying 'make it look like the previous one' so I had to design the whole page" (I37). Other interviewees mentioned edge cases that were not considered in the original specification—"there's edge cases and complications and … it's really hard to get information on what you have to do and in the end you gotta deliver something and so you make a decision and you really don't know if it's right or not" (I12).

4.1.5 *Developers Choose Technological Infrastructure*

Participants gave many examples of selecting technological infrastructure including programming languages, libraries, tools and service providers:

- "So, basically, we were deciding between, ugh, using C or C++ to do it" (I99).
- "What you find with that approach is that the tools that you bring in don't move very fast. Um, if you are subscribing to a service you know you've got the latest, bestest version of that tool" (I77).
- "I think one of the most difficult decisions that you have to make these days as a software developer is um, choosing a framework" (I78).
- "So I read about [this library] in the context of the mapreduce stuff ... and I did first use it for the mapreduce but I've used it consistently in other places" (I12).

While not every developer has control over their infrastructure, infrastructure elements including the technology stack clearly affect the code developers write and the decisions they can or must make (see Section 3.5).

4.2 Choice Architecture

4.2.1 *Developers Rarely Choose Between Several Options*

In Section 1, we defined a decision as "a conscious choice between two or more options". However, participants rarely articulated choices between several options. One of the few examples was, "do we return .Net data set, do we return JSUM or do we return XML?" (I50). Two explanations for this are apparent—1) developers rarely conceptualizing their work as decisions between more than two options; 2) developers have trouble remembering multiple options.

More often, multiple options were hinted at rather than listed—"there might be several different third-party, umm, libraries" (I44); "then I can just put a dot, and then do ctrl+space, and then I can have a look through the different functions that are available on that

object" (I44). This last example is particularly interesting as many IDEs provide some form of autocompletion (e.g. a list of possible methods), yet almost no one offered this as an example of a decision.

In some cases, prompting by the interviewer produced options that look suspiciously like post-hoc analysis. For example, Interviewee 7 explained two approaches to a problem—"standard linear equalization" and "first degree equalization"—but with prompting from the interviewer added a third—"histogram equalization." Similarly, when asked about options for when to refresh a display, Interviewee 41, having already talked about the callback architectural choice, observed, "I could've used a kind of a polling way of doing it rather than callback way of doing it, with the timer."

4.2.2 *Developers Sometimes Choose Between Two Options*

Participants sometimes, although not often, described choosing one of two options. For example, Interviewee 6 described how he "explored whether or not you create a, a authentication application which purely handles authentication for, across all these different applications, or do we integrate it into one of the apps, and all the other apps tap off that single app?" In other cases, the options were more like general preferences: "I tend to pass variables in class into methods via parameter, whereas some people just prefer to have a class level variable" (I31). Further examples already mentioned include push vs. pull information flow (architecture), composition versus inheritance (design) service versus tool (technology environment), and float versus double (implementation).

4.2.3 *Developers Face Dialectic Tension*

While participants provided few examples of clear choices between articulated alternatives, they often discussed a change emerging from broad tensions between two or more opposing forces, or *dialectic* [18, 30]. For example, many interviewees felt a pervasive tension between time and quality— "A lot of pressure to get things ahh performing, ahh so you might do something that's less efficient but works immediately as opposed to going online and researching it and doing something that's a bit more efficient but takes more time" (I7). Interviewee 22 agreed—"Do I refactor this? Do I make it nice? Or do I just write terrible code [laughs] and hope that it works." Interviwee 35 put it more bluntly: "Do it quickly and make it shit or do it well and stay up for the rest of the night" (I35).

Some interviewees specifically pointed to time as their primary constraint—"The other thing that constrains you, and this is the biggest in my opinion, is time" (I55). Several others specifically mentioned how refactoring decisions arose from the tension between time pressure and technical debt—"If the re-factoring will take one month and … might take two, but … the patch work would definitely just be a week's work ... you start to get their buy-in around unfortunately the cheap and fast option" (I65).

4.2.4 *Developers Choose to Take Action*

Most interviewee responses describe actions taken to solve problems with no reference to options or discrete choices, e.g.:

> *Well the thing is that, umm a lot of that code that we have written was exploratory could like you won't find anyone else uhh writing that anywhere, I mean like a lot of our job is to basically writing this hack through code so we weren't sure our self what we would do so we just created our own library. There wasn't like a this is how we are going to do it, there was no major plan it was just, the whole idea was to just keep everything as clean as possible, there were uhh no other options on the table.* (I61)

Often it was clear no options were considered—"It didn't even occur to me … until I showed it to someone else and they were like 'Oh, you've done it recursively' like" (I24). Meanwhile, Interviewee 22 explained how the choice of technology environment was sometimes made based on familiarity without considering options—"I have an affinity for NodeJS. My first decision was what language to use. Ah, I used NodeJS because it's easy."

5. TRIANGULATION AND DISCUSSION

In addition to coding decisions, participants described decisions related to software scope, design, architecture and the technologies and practices they use. They feel that their decisions are informed or constrained by many factors including their technology stack, coding standards, colleagues and a pervasive pressure to progress quickly and deliver on time. Some interviewees appear keenly aware of their frequent decision-making while others seem not to conceptualize their actions as decision-making.

Zannier et al. [32] found that when problems are more structured, software designers are more likely to evaluate a set of alternatives and attempt to select the optimal choice (rational decision making [13]), but when problems are less structured, software designers tend to explore a single, satisfactory option (naturalistic decision-making [11]). However, natural and rationalistic decision-making both involve *conscious* choices. Our results suggest something else entirely (Figure 4).

> Surprisingly, ***interviewees struggled to reconstruct coding decisions as choosing alternatives***. What could this mean? "You're making decisions a lot … these micro-decisions all the time and they're just hard to recall examples of, and just because they're everywhere and nowhere" (I12). Interviewee 22 agreed—"impulsive decisions ... I mean decisions that you make that you don't realise you'd made … [until] someone else points it out to you." This suggests that ***coding is more unconscious improvisation than conscious decision-making***.

Figure 4. Decision-Making or Unconscious Improvisation?

We know that code is replete with logical and organizational structures that have alternatives. We know developers have trouble articulating how and why they used certain structures instead of their alternatives. Many interviewees dodged repeated requests to detail choices in coding (see Appendix) by discussing broad considerations (e.g. time pressure) or non-coding decisions (e.g. architecture). Some interviewees explicitly said that they could not remember these "micro-decisions" because they are "impulsive"—i.e. unconscious. Unconscious improvisation is not decision-making.

These findings have several implications for research, practice and education. Decision-making is just one of many theoretical lenses for investigating, interpreting and explaining coding behavior. Other lenses include improvisation [14, 25, 33], heuristic search [23], problem solving [16], reflection-in-action [22] and writing [12]. Our results suggest a decision-making lens may be inappropriate for understanding coding behavior. This is consistent with previous findings that decision-making is insufficient to explain design thinking [1-3]. Choosing the right theoretical lens is crucial to understand how developers actually think while coding rather than how they appear to have thought, after the fact.

Meanwhile, developers and managers should maintain a balanced view of decision-making. While natural decision making leads to biases [10, 19, 29], rational decision-making only sometimes produces better decisions [4-7, 28] and always increases cognitive load (cf. [26]) and reduces speed. However, software quality may be im-

proved by developing tools that interrupt unconscious improvisation or natural decision-making at critical junctures where a more rational approach is warranted. Furthermore, software engineering curricula may be improved by including more exercises where students compare or create multiple approaches to a problem; for instance, devising three different architectures for the same system, reviewing two logically different but functionally equivalent code segments, or generating alternative wireframes for the same interface. Exercises like these should help students develop better solution space exploration habits (cf. [34]), but more research is needed to establish the effectiveness of this approach.

These recommendations should be weighed against several limitations. Since there is no representative list of the world's developers, we used convenience sampling; therefore, our results cannot be statistically generalized to a broader population. Analytically, Leximancer is limited by the possibility of missing important concepts because the text is not read in its natural order, while manual coding is limited by how much the research can practically read. To mitigate these limitations, we read the text in order during manual coding and evaluated all of the data using Leximancer. Two further threats to validity are memory and desirability effects; that is, participants may misremember some details or try to rationalize their decision processes and actions. More generally, had we known a priori that the results would suggest that code-focused decisions are often unconscious, we would have designed the study differently. Future work may include direct observation, simulation or think-aloud protocol studies to further investigate the cognitive processes underlying coding behavior.

6. CONCLUSION

In summary, we set out to develop an ontology of coding decisions from interviews of professional developers. However, we discovered that developers were rarely capable of reconstructing *code-focused* decisions between *two or more alternatives*. We therefore refocused on two related research questions: 1) What kinds of decisions do programmers make, and 2) What are the choice architectures of those decisions?

Our analysis suggests that developers make at least four different kinds of decisions:

1. Which tools and infrastructure to use (e.g. Node.js)
2. Scope decisions (e.g. to omit a feature)
3. Architecture decisions (e.g. pull information flow)
4. Design decisions (e.g. storing data in an array)
5. Low-level implementation properties (e.g. variable names)

Moreover, our analysis suggests that developers seldom consciously consider several alternatives and select the best. More often, developers adopt a unidirectional choice architecture where they simply decide to do something or take some action.

In other words, while *code* appears full of decisions, *coders* are often not conscious of their choices or alternatives. This suggests that decision-making is a poor theoretical lens for investigating coding behaviors and thought processes. Further investigation of developer improvisation and cognition is clearly needed.

7. ACKNOWLEDGMENTS

We would like to thank all of those who participated in this study.

8. APPENDIX: INTERVIEW GUIDE

8.1 Introductory Questions

- What kind of team do you work on? What's it like?
- What is your role?
- What are you currently developing?

8.2 Main Questions

- Can you think of a really difficult decision you had to make while programming? What was it? What were the options? What did you choose? Why?
- Were there other solutions you could have picked but didn't? Why or why not?
- Have any of your coding decisions that turned out unexpectedly well? What was it? What were the options? What did you choose? Why?
- Have you ever had a programming decision that came back to haunt you? What was it? What were the options? What did you choose? Why?
- Do you feel that you made a mistake or that you acted reasonably at the time with what you had?
- Can you think of a time when you were programming and had to make a decision that should have been made by someone else? What was it? What were the options? What did you choose? Why?
- Can you think of a time when you were programming and had to make a decision that should have been made beforehand? What was it? What were the options? What did you choose? Why?
- Can you think of a time when you were programming and had to make a decision that had to be approved by someone else? What was it? What were the options? What did you choose? Why?
- Can you think of a time when you were programming and had to make a decision where you had to stop because you didn't have enough information? What information did you need? What was it? What were the options? What did you choose? Why?
- If you come to a dead end, what do you do? Who do you consult?
- Can you think of a time when you were programming and made a decision based on a best practice (e.g. encapsulation), design pattern or common wisdom?
- What was it based on and where did you learn it? What was it? What were the options? What did you choose? Why?
- Can you think of a specific programming decision that was constrained by something like an coding standard, technology stack or process? What was it? What were the options? What did you choose? Why?
- Are these (standards etc.) ever a source of frustration or difficulties?
- Can you give me a specific example of a time you intentionally disregarded a best practice? What was it? What were the options? What did you choose? Why?
- Have you ever discovered issues in your code and decided to ignore them? What was the issue? Why was it bad? Why did you ignore it?
- How much freedom do you have over your programming decisions?
- Can you think of a time when you were programming and made a decision based on something you read online? What was it? What were the options? What did you choose? Why?
- Can you think of a time when you were programming and made a decision based on something you read in a book? What was it? What were the options? What did you choose? Why?
- Can you think of a time when you were programming and made a decision based on something you learned in some kind of degree or diploma training? What was it? What were the options? What did you choose? Why?

- Is there a particular kind of decision that you often struggle with?
- Do you find commenting code helps your decision making?

8.3 Possible Follow-up Questions

- Are you expected to write unit tests? Do you? Why (not)?
- Do you do pair programming? Have you ever been pair programming and disagreed about a decision?
- Do you have code reviews? Tell me about a decision that was questioned during a code review?
- Have you ever had trouble deciding where to start? What to implement first?
- Where did you start?
- What helps you find your starting point?
- What tools do you use? Why?
- What do you think of the communication about decision making between developers and management?
- How does code ownership work in your team?
- Have you ever had to make decisions about other people's code? Would you give me an example?

8.4 Concluding Questions

- Do you make any other kind of programming decisions that we haven't talked about?
- Is there anything else you'd like to tell me, or anything I should know?

9. REFERENCES

[1] Cross, N. 1997. Descriptive models of creative design: application to an example. *Design Studies*. 18, (Oct. 1997), 427–440.

[2] Cross, N. 2011. *Design Thinking: Understanding How Designers Think and Work*. Berg.

[3] Cross, N., Dorst, K. and Roozenburg, N. 1992. *Research in Design Thinking*. Delft University Press.

[4] Gigerenzer, G. 1991. How to make cognitive illusions disappear: Beyond "Heuristics and Biases." *European Review of Social Psychology*. 2, (1991), 83–115.

[5] Gigerenzer, G. 2008. Why heuristics work. *Perspectives on Psychological Science*. 3, 1 (Jan. 2008), 20–29.

[6] Gigerenzer, G. and Todd, P.M. 1999. *Simple heuristics that make us smart*. Oxford University Press.

[7] Gigerenzer, G., Hertwig, R., Hoffrage, U. and Sedlmeier, P. 2008. Cognitive illusions reconsidered. *Handbook of Experimental Economics Results*. R.P. Charles and L.S. Vernon, eds. Elsevier. 1018–1034.

[8] Jansson, D.G. and Smith, S.M. 1991. Design fixation. *Design Studies*. 12, 1 (1991), 3–11.

[9] Johnson, E.J., Shu, S.B., Dellaert, B.G.C., Fox, C., Goldstein, D.G., Häubl, G., Larrick, R.P., Payne, J.W., Peters, E., Schkade, D., Wansink, B. and Weber, E.U. 2012. Beyond nudges: Tools of a choice architecture. *Marketing Letters*. 23, 2 (May 2012), 487–504.

[10] Kahneman, D. 2011. *Thinking, Fast and Slow*. Penguin UK.

[11] Klein, G. 1999. *Sources of Power: How People Make Decisions*. The MIT Press.

[12] Knuth, D.E. 1992. *Literate Programming*. Stanford University Center for the Study of Language and Information.

[13] Luce, D. and Raiffa, H. 1957. *Games and Decisions: Introduction and Critical Survey*. Wiley.

[14] Magni, M., Maruping, L., Hoegl, M. and Proserpio, L. 2008. Improvisation and performance in software development teams: The Role of geographic dispersion. *Proceedings of the International Conference on Information Systems*. AIS.

[15] Mohanani, R., Ralph, P. and Shreeve, B. 2014. Requirements fixation. *Proceedings of the International Conference on Software Engineering*. IEEE. 895–906.

[16] Newell, A. and Simon, H. 1972. *Human Problem Solving*. Prentice-Hall, Inc.

[17] Purcell, A.T., Williams, P., Gero, J.S. and Colbron, B. 1993. Fixation effects: do they exist in design problem solving? *Environment and Planning B*. 20, (1993), 333–333.

[18] Ralph, P. 2015. Developing and evaluating software engineering process theories. *Proceedings of the International Conference on Software Engineering*. 20–31.

[19] Ralph, P. 2011. Toward a Theory of Debiasing Software Development. *Research in Systems Analysis and Design: Models and Methods: 4th SIGSAND/PLAIS EuroSymposium 2011*. S. Wrycza, ed. Springer. 92–105.

[20] Ralph, P. and Kelly, P. 2014. The Dimensions of Software Engineering Success. *Proceedings of the International Conference on Software Engineering*. ACM. 24–35.

[21] Saldana, J. 2012. *The Coding Manual for Qualitative Researchers*. SAGE Publications.

[22] Schön, D.A. 1983. *The reflective practitioner: how professionals think in action*. Basic Books.

[23] Simon, H.A. 1996. *The Sciences of the Artificial*. MIT Press.

[24] Smith, A.E. and Humphreys, M.S. 2006. Evaluation of unsupervised semantic mapping of natural language with Leximancer concept mapping. *Behavior Research Methods*. 38, 2 (May 2006), 262–279.

[25] Suchman, L. 2007. *Human-machine reconfigurations: Plans and situated actions*. Cambridge University Press.

[26] Sweller, J. 1988. Cognitive Load During Problem Solving: Effects on Learning. *Cognitive Science*. 12, 2 (Apr. 1988), 257–285.

[27] Thaler, R.H. and Sunstein, C.R. 2008. *Nudge*. Yale University Press.

[28] Todd, P.M. and Gigerenzer, G. 2001. Putting naturalistic decision making into the toolbox. *Journal of Behavioral Decision Making*. 14, 5 (2001), 381–383.

[29] Tversky, A. and Kahneman, D. 1974. Judgment under Uncertainty: Heuristics and Biases. *Science*. 185, 4157 (1974), 1124–1131.

[30] Van de Ven, A.H. and Poole, M.S. 1995. Explaining development and change in organizations. *The Academy of Management Review*. 20, 3 (1995), 510–540.

[31] Zannier, C. and Maurer, F. 2006. Foundations of Agile Decision Making from Agile Mentors and Developers. *Extreme Programming and Agile Processes in Software Engineering*. Springer Berlin Heidelberg. 11–20.

[32] Zannier, C., Chiasson, M. and Maurer, F. 2007. A model of design decision making based on empirical results of interviews with software designers. *Information and Software Technology*. 49, 6 (2007), 637–653.

[33] Zheng, Y., Venters, W. and Cornford, T. 2011. Collective agility, paradox and organizational improvisation: the development of a particle physics grid. *Information Systems Journal*. 21, 4 (2011), 303–333.

[34] 2012. Improving coverage of design in information systems education. *Proceedings of the 2012 International Conference on Information Systems*. AIS.

[35] 2015. *OED Online*. Oxford University Press.